MANUAL TECNICO PROFESIONAL

JuegoTanques

Arquitectura, configuracion, ejecucion, despliegue, operacion y mantenimiento del sistema multijugador para Android y web.



Version del documento	1.0
Fecha	28 de octubre de 2026
Estado	Basado en el repositorio compilable actual
Plataformas	Windows, navegador y Android

Control del documento

Campo	Detalle
Proposito	Servir como referencia tecnica para desarrollo, pruebas, soporte, despliegue y continuidad del proyecto.
Alcance	Backend .NET, SQL Server, API REST, SignalR, frontend Angular/Ionic, motor de juego Canvas y empaquetado Android.
Audiencia	Desarrolladores, docentes, evaluadores, responsables de infraestructura y personal de soporte.
Fuente	Inspeccion directa del codigo y configuracion existentes en el repositorio JuegoTanques.
Exclusiones	No incluye credenciales reales, secretos, contraseñas ni instrucciones para desactivar controles de seguridad del sistema operativo.

Nota de confidencialidad

El repositorio contiene valores sensibles en archivos de configuracion y una cadena de conexion generada por scaffolding. Este manual los reemplaza por marcadores seguros y recomienda migrarlos a secretos de usuario o variables de entorno.

Resumen ejecutivo

JuegoTanques es una aplicacion de combate de tanques con modos offline y multijugador. El cliente se implementa como una aplicacion Angular standalone con Ionic, usa Canvas 2D para el motor de juego y se empaqueta en Android mediante Capacitor. El backend utiliza ASP.NET Core, autenticacion JWT, SignalR para comunicacion en tiempo real, Entity Framework Core y SQL Server.

- El modo offline ejecuta fisica, colisiones, IA, proyectiles, rondas y power-ups localmente.
- El modo online comparte salas, estados de jugadores, disparos, impactos, rondas y resultados mediante SignalR.
- La API REST gestiona autenticacion, perfiles, catalogo, inventario, presets, tienda, ranking y progreso.
- La aplicacion Android consume la API publica mediante un tunel HTTPS de ngrok durante la etapa de pruebas.

Tabla de contenido

Control del documento 2

Resumen ejecutivo 2

Tabla de contenido 3

1. Vision general del sistema 6

1.1 Objetivos funcionales . . . 6

1.2 Arquitectura logica . . . 6

1.3 Principios de diseño observados . . . 6

2. Estructura del repositorio 7

2.1 Componentes frontend principales . . . 7

2.2 Deuda estructural identificada . . . 7

3. Tecnologias y requisitos 8

3.1 Requisitos del equipo de desarrollo . . . 8

4. Configuracion segura 9

4.1 Backend . . . 9

4.2 Cliente . . . 9

4.3 CORS y HTTPS . . . 9

5. Backend ASP.NET Core 10

5.1 Composicion del host . . . 10

5.2 Capas . . . 10

5.3 Endpoint de salud . . . 10

6. Modelo de datos 11

6.1 Inicializacion de un jugador . . . 11

7. API REST 12

7.1 Autenticacion . . . 12

7.2 Operaciones autenticadas de juego . . . 12

7.3 Controladores CRUD . . . 12

8. Autenticacion y sesion 13

8.1 Flujo de registro . . . 13
8.2 JWT . . . 13
8.3 Persistencia movil . . . 13
9. Multijugador con SignalR 14
9.1 Metodos invocados por el cliente . . . 14
9.2 Eventos recibidos . . . 14
9.3 Autoridad y sincronizacion . . . 14
10. Frontend Angular e Ionic 15
10.1 Arranque . . . 15
10.2 Rutas . . . 15
10.3 Estado de juego . . . 15
10.4 Ciclo de orientacion . . . 15
11. Motor de juego offline 16
11.1 Bucle y renderizado . . . 16
11.2 Reglas actuales . . . 16
11.3 Equipamiento . . . 16
12. Motor de juego online 17
12.1 Flujo de un disparo . . . 17
12.2 Modos . . . 17
13. Ajustes, audio y feedback 18
14. Ejecucion local desde Visual Studio 19
14.1 Preparacion . . . 19
14.2 Inicio del backend . . . 19
14.3 Inicio del frontend web . . . 19
15. Acceso externo con ngrok 20
15.1 Flujo recomendado . . . 20
15.2 Consideraciones . . . 20
16. Compilacion del APK 21
16.1 Proceso reproducible . . . 21

16.2 Firma de produccion . . . 21

16.3 Compatibilidad . . . 21

17. Pruebas y validacion **22**

17.1 Matriz minima . . . 22

17.2 Comandos . . . 22

18. Operacion y diagnostico **23**

18.1 Logs . . . 23

19. Seguridad y recomendaciones **24**

19.1 Orden de endurecimiento recomendado . . . 24

20. Mantenimiento y evolucion **25**

20.1 Procedimiento de cambio . . . 25

20.2 Roadmap tecnico sugerido . . . 25

21. Apendices **26**

A. Puertos y direcciones . . . 26

B. Catalogo local del motor . . . 26

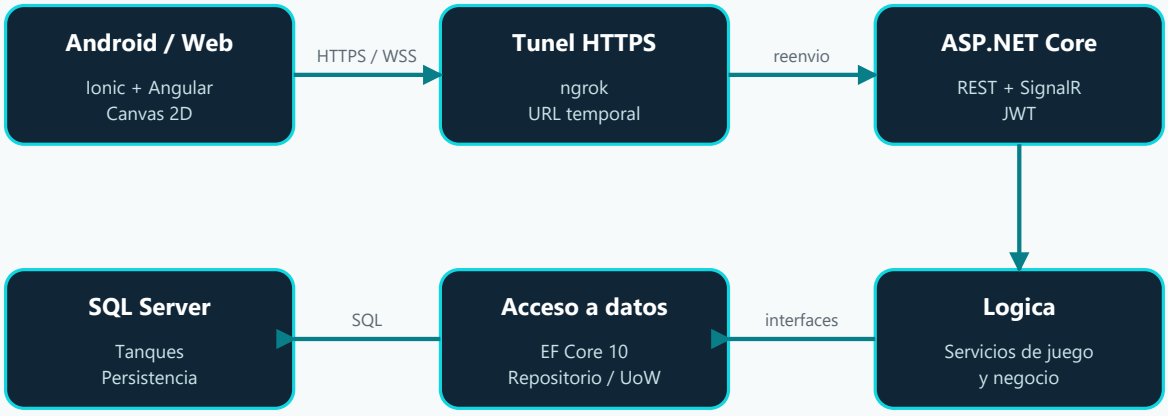
C. Glosario . . . 26

1. Vision general del sistema

1.1 Objetivos funcionales

- Registrar usuarios con verificacion de correo y entregar equipo inicial.
- Mantener sesion local en el dispositivo mientras el JWT sea valido.
- Permitir comprar, equipar y guardar configuraciones de tanque y arma.
- Ejecutar partidas offline configurables contra bots.
- Crear y unir jugadores a salas online con modos individual o por equipos.
- Registrar progreso, monedas, experiencia, ranking y resultados de partida.

1.2 Arquitectura logica



Flujo principal: cliente -> tunel -> API/Hub -> negocio -> datos

La arquitectura separa presentacion, transporte, reglas de negocio y persistencia. En tiempo real, el cliente abre una conexion WebSocket/SignalR autenticada con el mismo JWT usado por REST. Durante las pruebas externas, ngrok publica el puerto HTTP local del backend como HTTPS/WSS.

1.3 Principios de diseño observados

- Frontend standalone con carga diferida para la mayoria de pantallas.
- Inyeccion de dependencias nativa de Angular y ASP.NET Core.
- Capas de dominio, acceso a datos, logica de negocio y API.
- Estado reactivo en el cliente mediante signals y computed values.
- Servidor autoritativo para impactos, vidas, rondas y recompensas online.

2. Estructura del repositorio

Proyecto / carpeta	Responsabilidad tecnica
JuegoTanques.API	Host ASP.NET Core, controladores REST, JWT, correo, SignalR, salas, progreso y configuracion.
JuegoTanques.Dominio	Entidades EF, DTO, interfaces de repositorio y logica de negocio, perfiles de AutoMapper.
JuegoTanques.AccesoDatos	DbContext, repositorio generico y unidad de trabajo sobre SQL Server.
JuegoTanques.LogicaNegocio	Implementaciones CRUD y reglas de negocio de las entidades persistentes.
JuegoTanques.Utilitarios	Contrato generico de respuesta utilizado por distintas capas.
JuegoTanquesFront	Aplicacion Angular/Ionic, pantallas, servicios, motor Canvas, assets y proyecto Android Capacitor activo.
android (raiz)	Copia Android adicional. No corresponde al webDir configurado en JuegoTanquesFront; debe considerarse candidata a eliminacion tras validar que no se use.
JuegoTanques	Proyecto de biblioteca minimo sin responsabilidad funcional visible; candidato a limpieza si no existe dependencia externa.

2.1 Componentes frontend principales

Area	Elementos
Pantallas	login, signup, inicio, offline, tienda, salas, sala-espera, ranking, perfil y settings.
Motor	GameMap, Tank, Bullet, BotAI, ItemManager, AimTrajectory, GamePlayComponent y OnlineGameComponent.
Servicios	AuthService, interceptor JWT, SignalrService, JuegoStore, AjustesService, FeedbackService y UsuarioService.
Seguridad de rutas	authGuard protege areas privadas; guestGuard evita volver al login cuando la sesion es valida.
Modelos	Contratos de autenticacion, respuestas, juego, catalogo y datos locales de tanques, armas, mapas y power-ups.

2.2 Deuda estructural identificada

- Existe un interceptor duplicado bajo app/interceptors; la aplicacion registra el ubicado en app/services.
- Existe un archivo ReactComponent.jsx dentro de guards sin integracion con Angular.
- Hay dos arboles Android; el flujo Capacitor actual usa JuegoTanquesFront/android.
- El proyecto JuegoTanques/Class1.cs no aporta comportamiento al sistema actual.
- Algunos componentes mantienen imports Ionic no utilizados y generan advertencias NG8113.

3. Tecnologias y requisitos

Capa	Tecnologia / version	Funcion
Backend	.NET 10 / ASP.NET Core	API, DI, autenticacion, SignalR y host.
Persistencia	EF Core 10.0.12 / SQL Server	Mapeo relacional y acceso a datos.
Frontend	Angular 22.1.7 / TypeScript 6	UI standalone y logica del cliente.
UI movil	Ionic Angular 9	Controles, navegacion y ciclo de vida movil.
Runtime movil	Capacitor 8.5.3	Puente Android y plugins nativos.
Tiempo real	SignalR 10.0.11	Salas y sincronizacion de partidas.
Android	SDK 24 minimo, target/compile 36	APK Android 7.0 o superior.
Build Android	Gradle 8.14.3 / JDK de Android Studio	Compilacion y firma debug.
Exposicion temporal	ngrok HTTPS	Acceso desde dispositivos externos durante pruebas.

3.1 Requisitos del equipo de desarrollo

- Windows 11 con Visual Studio y carga de trabajo ASP.NET y desarrollo web.
- .NET SDK compatible con net10.0.
- SQL Server Express o superior y una base Tanques con el esquema esperado.
- Node.js, npm y Angular CLI compatibles con el package-lock del proyecto.
- Android Studio con SDK 36 y JDK integrado para compilar el APK.
- Cuenta y agente ngrok para pruebas fuera de la red local.

4. Configuracion segura

4.1 Backend

Clave	Descripcion	Recomendacion
ConnectionStrings:DefaultConnection	Conexion a SQL Server.	Usar User Secrets en desarrollo y variable de entorno en despliegue.
Jwt:Clave	Secreto HMAC de firma.	Minimo 32 bytes aleatorios. No versionar.
Jwt:Emisor / Audiencia	Identidad del emisor y cliente esperado.	Mantener consistentes entre generacion y validacion.
Jwt:HorasValidez	Vigencia de sesion, actualmente 720 horas.	Revisar segun politica; para produccion considerar refresh tokens.
Correo:*	Servidor SMTP y credenciales.	Rotar las credenciales expuestas y moverlas fuera de appsettings.json.

```
dotnet user-secrets init --project JuegoTanques.API
dotnet user-secrets set "Jwt:Clave" "<SECRETO_ALEATORIO>" --project JuegoTanques.API
dotnet user-secrets set "ConnectionStrings:DefaultConnection" "<CONEXION_SQL>" --project JuegoTanques.API
```

Hallazgo critico

JuegoTanquesContext.OnConfiguring contiene una conexion local embebida y appsettings.json contiene material sensible. Aunque la API recibe una conexion por DI, OnConfiguring puede actuar como respaldo. Debe eliminarse la cadena literal y conservarse solo la configuracion externa.

4.2 Cliente

environment.ts y environment.prod.ts apuntan actualmente a la misma URL publica de ngrok. Esta direccion es temporal y debe actualizarse cuando el tunel cambie. El interceptor agrega Authorization: Bearer para llamadas a la API y el encabezado ngrok-skip-browser-warning para evitar la pagina intermedia de ngrok.

```
export const environment = {
production: true,
baseUrl: 'https://<subdominio>.ngrok-free.dev/api'
};
```

4.3 CORS y HTTPS

- La politica PermitirIonic autoriza localhost, puertos 8000/8100/8101 y capacitor://localhost.
- No se usa AllowAnyOrigin junto con credenciales, lo cual evita una configuracion CORS invalida.
- El backend local se sirve por HTTP y ngrok termina HTTPS durante la demostracion.
- Para produccion debe configurarse proxy inverso, forwarded headers, HTTPS permanente y dominio estable.

5. Backend ASP.NET Core

5.1 Composicion del host

Program.cs registra controladores, OpenAPI en desarrollo, SignalR, DbContext, AutoMapper, repositorios, unidades de trabajo, servicios de negocio, autenticacion JWT, autorizacion, CORS y memoria cache. SalasServicio es singleton porque mantiene el estado efimero de las salas; los servicios que usan DbContext son scoped.

Servicio	Ciclo de vida	Responsabilidad
JuegoTanquesContext	Scoped	Sesion EF Core por solicitud.
IUnidadTrabajoEF	Scoped	Repositorio y confirmacion transaccional.
Servicios LN	Scoped	Reglas CRUD por entidad.
AuthServicio	Scoped	Registro, verificacion, login y equipo inicial.
JuegoServicio	Scoped	Resumen, inventario, presets, compras y ranking.
ProgresoServicio	Scoped	Resultados, XP, nivel y recompensas.
SalasServicio	Singleton	Estado online, rondas, disparos y power-ups.
JwtServicio	Singleton	Generacion de tokens firmados.

5.2 Capas

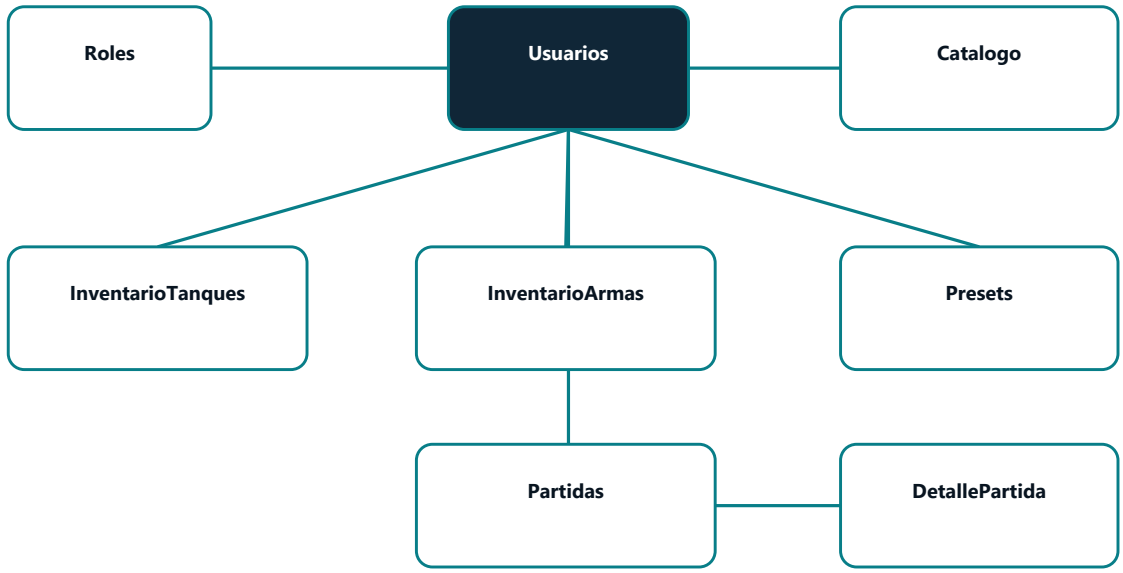
- API: adapta HTTP/SignalR a casos de uso y obtiene identidad desde claims.
- LogicaNegocio: implementa contratos de dominio para operaciones persistentes.
- Dominio: define entidades, DTO e interfaces, evitando dependencia directa de UI.
- AccesoDatos: encapsula EF Core mediante contexto, repositorio y unidad de trabajo.

5.3 Endpoint de salud

```
GET /api/health
Respuesta esperada: { "ok": true, "servicio": "JuegoTanques.API" }
```

6. Modelo de datos

Modelo relacional simplificado



Entidad	Clave	Datos relevantes
Usuario	usuario_id	username, email, password_hash, rol, nivel, experiencia, monedas, fecha.
Role	rol_id	Nombre unico del rol.
Tanque	tanque_id	nombre, vida, velocidad, blindaje, precio.
Arma	arma_id	nombre, dano, cadencia, velocidad del proyectil, precio.
InventarioTanque	usuario_id + tanque_id	Relacion de propiedad y fecha de adquisicion.
InventarioArma	usuario_id + arma_id	Relacion de propiedad y fecha de adquisicion.
EquipamientosPreguardados	preset_id	Slot, tanque, arma, nombre y bandera activa.
Partida	partida_id	Modo, inicio, fin y duracion.
DetallePartidaJugador	detalle_id	Jugador, posicion, destruidos, dano, XP y monedas.

6.1 Inicializacion de un jugador

Al confirmar el codigo de correo, AuthServicio crea el Usuario dentro de una transaccion, asigna el rol Cliente, 500 monedas, nivel 1, tanque base, arma base y un preset activo. Este comportamiento depende de que existan previamente el rol y los catalogos con identificadores base.

7. API REST

7.1 Autenticacion

Metodo	Ruta	Uso
POST	/api/Auth/Login	Login por correo o username; devuelve JWT y perfil minimo.
POST	/api/Auth/Registrar	Valida datos, crea registro pendiente y envia codigo.
POST	/api/Auth/VerificarCorreo	Valida codigo, crea usuario, inventario y sesion.
POST	/api/Auth/ReenviarCodigo	Regenera codigo respetando tiempo minimo y expiracion.

7.2 Operaciones autenticadas de juego

Metodo	Ruta	Responsabilidad
GET	/api/Jugador/Resumen	Perfil, nivel, experiencia, monedas y equipo activo.
GET	/api/Jugador/Inventario	IDs de tanques y armas adquiridos.
GET/POST	/api/Jugador/Presets	Listar o guardar configuraciones.
PUT	/api/Jugador/Presets/{id}/Activar	Marcar un preset como activo.
PUT	/api/Jugador/Equipar	Equipar tanque/arma y sincronizar preset.
GET	/api/Catalogo	Catalogo completo de tanques y armas.
POST	/api/Tienda/Comprar	Validar saldo, propiedad y registrar compra.
GET	/api/Ranking	Clasificacion de jugadores.
POST	/api/Partida/FinalizarOffline	Registrar resultado y progreso offline.

7.3 Controladores CRUD

Arma, Tanque, Role, Usuario, Partida, DetallePartidaJugador, InventarioTanque y EquipamentosPreguardados exponen operaciones CRUD convencionales. Actualmente varias de estas rutas no muestran Authorize a nivel de controlador, por lo que deben protegerse o limitarse por rol antes de un despliegue real.

> **Contrato de respuesta**
>
> La mayoria de servicios devuelven un contenedor de respuesta con indicador de exito, mensaje y dato. El cliente extrae el dato y convierte fallos HTTP o de negocio en mensajes de UI.

8. Autenticacion y sesion

8.1 Flujo de registro

- El cliente envia nombre, correo y contrasena.
- El backend valida formato, duplicados y longitud minima.
- La contrasena se deriva con PBKDF2-SHA256, sal aleatoria y 100 000 iteraciones.
- Se genera un codigo numerico de seis digitos y se guarda en IMemoryCache por 10 minutos.
- La comparacion del codigo y del hash usa comparacion de tiempo constante.
- Al verificar, se crea el usuario y su equipo base dentro de una transaccion SQL.

8.2 JWT

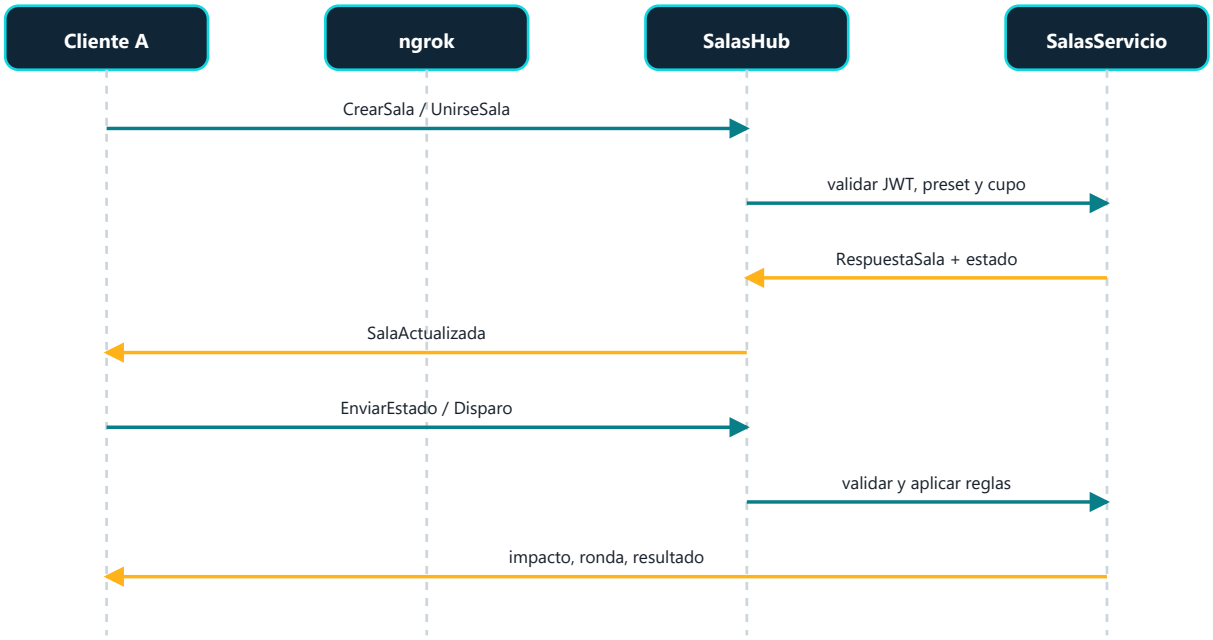
El token contiene NameIdentifier, Name y Role; se firma con HMAC-SHA256 y valida emisor, audiencia, firma y expiracion. SignalR recupera el token desde access_token para la ruta /salasHub. La vigencia actual es de 720 horas.

8.3 Persistencia movil

AuthService guarda la respuesta de sesion en localStorage bajo tienda-sesion. Al iniciar, guestGuard redirige directamente a /home cuando el token no ha expirado. authGuard bloquea las rutas privadas y el interceptor elimina la sesion tras un 401 no relacionado con autenticacion. Cerrar o minimizar la app no borra localStorage; limpiar datos, desinstalar o expirar el JWT si requiere autenticarse nuevamente.

> **Limitacion actual**
>
> No existe refresh token ni revocacion central. Para produccion se recomienda access token corto, refresh token rotatorio, cierre de sesion servidor y almacenamiento nativo seguro para credenciales de renovacion.

9. Multijugador con SignalR



9.1 Metodos invocados por el cliente

Metodo Hub	Funcion
ListarSalas	Devuelve salas disponibles.
CrearSala / UnirseSala	Crea o incorpora al usuario con su preset.
SalirSala	Retira conexion y actualiza el grupo.
IniciarPartida / EstoyListo	Coordina inicio y preparacion de jugadores.
EnviarEstado	Publica posicion, angulos y HP.
EnviarDisparo / RegistrarImpacto	Registra proyectil y valida dano del objetivo.
CrearPowerUp / RecogerPowerUp	Sincroniza objetos compartidos.
LanzarZona / AplicarVeneno	Gestiona habilidad de area y ticks validados.

9.2 Eventos recibidos

SalaActualizada, ListaSalasActualizada, PartidaIniciada, EstadoRemoto, DisparoRemoto, ZonaRemota, RondaIniciada, RondaTerminada, PartidaTerminada, JugadorImpactado, JugadorMurio, JugadorReaparecio, PowerUpAparecio, PowerUpRecogido, DisparoConsumido y JugadorSalio.

9.3 Autoridad y sincronizacion

- El cliente envia estado aproximadamente cada 50 ms durante la partida.
- Los clientes interpolan posiciones y angulos remotos para suavizar movimiento.
- La victima detecta colision con proyectil remoto y solicita RegistrarImpacto.
- El servidor conserva la bala registrada, valida atacante, equipo y dano, y emite el resultado.
- El host de sala genera power-ups; el servidor valida su registro y recoleccion.
- SalasServicio persiste estado de sala en memoria; reiniciar la API elimina salas activas.

10. Frontend Angular e Ionic

10.1 Arranque

main.ts inicia AppComponent con deteccion de cambios coalescida, IonicRouteStrategy, preloading de modulos, HttpClient con interceptor funcional y el enrutador. Las pantallas standalone reducen la necesidad de NgModules.

10.2 Rutas

Ruta	Pantalla	Proteccion
/login, /signup	Autenticacion y registro	guestGuard
/home	Navegacion principal	authGuard
/offline, /game	Configuracion y partida offline	authGuard
/salas, /sala-espera, /game-online	Flujo multijugador	authGuard
/store, /ranking, /perfil, /settings	Servicios del jugador	authGuard

10.3 Estado de juego

JuegoStore centraliza resumen, catalogo, inventario, presets y ranking en signals. tanqueEquipado y armaEquipada son valores computed derivados del preset activo y del catalogo. cargarTodo ejecuta la carga necesaria para Inicio y Offline.

10.4 Ciclo de orientacion

- Menus, configuracion y seleccion offline solicitan orientacion vertical.
- Partidas offline y online bloquean horizontal al entrar.
- Al destruir o abandonar la partida se restablece vertical.
- En navegadores sin API de orientacion, los errores se ignoran de forma controlada.

11. Motor de juego offline

11.1 Bucle y renderizado

GamePlayComponent utiliza requestAnimationFrame. En cada frame actualiza entradas, habilidad, bots, objetos, balas, colisiones, municion y camara; despues dibuja mapa, zonas, objetos, tanques, barras de vida, proyectiles y trayectoria de apuntado. El mundo logico trabaja en 1280 x 720 y la camara adapta la visualizacion movil.

Modulo	Responsabilidad
GameMap	Matriz de colisiones, paredes y dibujo del escenario.
Tank	Posicion, movimiento, angulo, torreta, vida y sprites.
Bullet	Velocidad, rebote/colision, perforacion, rango y dibujo.
BotAI	Movimiento, persecucion, disparos y dificultad.
ItemManager	Aparicion, recoleccion y efectos temporales.
AimTrajectory	Prediccion grafica de la trayectoria del proyectil.

11.2 Reglas actuales

- Partidas configurables por dificultad, numero de bots, rondas y mapa.
- Municion limitada con recarga progresiva dependiente del arma.
- Scout: dash; Juggernaut: zona de veneno/ralentizacion; Vanguardia: proyectil perforante.
- Power-ups de curacion, escudo y sobrecarga; penalizaciones de aceite, oruga y falla.
- Final de ronda por destruccion, muerte del jugador o tiempo agotado.

11.3 Equipamiento

OfflineComponent aplica primero el equipo ya cargado en JuegoStore y luego actualiza desde la API. Tras el await se fuerza deteccion de cambios para evitar que Ionic muestre tanque y arma solo despues de tocar la pantalla. La partida recibe configuracion, equipo y bots mediante Router state.

12. Motor de juego online

OnlineGameComponent reutiliza GameMap, Tank, Bullet y AimTrajectory, pero mantiene un mapa de OnlinePlayer y wrappers de proyectiles con propietario, equipo e indicador de impacto solicitado. El jugador local usa fisica inmediata; los remotos se interpolan hacia el ultimo estado recibido.

12.1 Flujo de un disparo

- El cliente valida ronda, estado, municion y efectos.
- Crea el proyectil local y llama EnviarDisparo con un ID unico.
- El servidor registra la bala y la publica al grupo.
- Un cliente victima detecta la colision y llama RegistrarImpacto.
- El servidor aplica reglas, consume la bala y emite JugadorImpactado o JugadorMurio.
- Los clientes actualizan HP, vidas, feedback haptico y resultado de ronda.

12.2 Modos

- Todos contra todos: cada jugador restante constituye un bando independiente.
- Equipos: el dano aliado se ignora y la ronda termina cuando queda un equipo con vidas.
- Resultados: PartidaTerminada entrega posiciones, kills, XP, monedas, nivel y subida de nivel.

Criterio de red

El sistema usa actualizacion frecuente y validacion parcial del servidor, adecuada para una demostracion academica. Para produccion competitiva se recomienda simulacion autoritativa completa, limites de frecuencia, reconciliacion, timestamps y validacion anti-cheat.

13. Ajustes, audio y feedback

Ajuste	Aplicacion
Volumen de efectos	Escala el gain del Web Audio API entre 0 y 100%.
Sensibilidad	Controla la interpolacion angular de la torreta en ambos modos.
Apuntado asistido	Permite disparar al enemigo cercano al soltar el stick sin direccion.
Vibracion	Activa Haptics Light, Medium o Heavy segun disparo/impacto.
Mostrar FPS	Presenta un contador calculado cada segundo.

AjustesService persiste la configuracion en localStorage, mezcla valores predeterminados y normaliza rangos. FeedbackService genera tonos cortos con Web Audio sin archivos externos y usa @capacitor/haptics en Android. Si el navegador no admite una capacidad, la excepcion se absorbe y el juego continua.

14. Ejecucion local desde Visual Studio

14.1 Preparacion

- Abrir JuegoTanques.slnx en Visual Studio.
- Restaurar paquetes NuGet y npm.
- Confirmar que SQL Server esta activo y que la base Tanques contiene roles, catalogo y tablas.
- Configurar secretos de JWT, correo y conexion sin escribirlos en el repositorio.
- Seleccionar JuegoTanques.API como proyecto de inicio.

14.2 Inicio del backend

El perfil HTTP escucha en http://localhost:5140 y el perfil HTTPS tambien expone https://localhost:7123. Para el tunel usado por la app debe iniciarse el perfil HTTP o apuntar ngrok al puerto efectivo mostrado por Visual Studio.

```
# Alternativa por terminal
dotnet restore JuegoTanques.slnx
dotnet run --project JuegoTanques.API --launch-profile http
```

14.3 Inicio del frontend web

```
cd JuegoTanquesFront
npm install
npm start
# Abrir http://localhost:8100
```

DLL bloqueada durante compilacion

Si Visual Studio ya esta ejecutando JuegoTanques.API, sus DLL permanecen abiertas y una segunda compilacion puede fallar al copiarlas. Detenga la depuracion con el boton rojo, espere a que termine el proceso y vuelva a iniciar. No es necesario desactivar la seguridad de Windows.

15. Acceso externo con ngrok

15.1 Flujo recomendado

- Iniciar SQL Server.
- Iniciar JuegoTanques.API desde Visual Studio en http://localhost:5140.
- Abrir una terminal separada y publicar exactamente ese puerto con ngrok.
- Copiar la URL HTTPS entregada por ngrok en ambos environment del frontend.
- Reconstruir y sincronizar la aplicacion Android.
- Verificar /api/health desde una red externa antes de distribuir el APK.

```
ngrok http 5140

# Comprobacion externa
https://<subdominio>.ngrok-free.dev/api/health
```

15.2 Consideraciones

- La URL gratuita puede cambiar al reiniciar ngrok salvo que se use un dominio reservado.
- El equipo anfitrion debe permanecer encendido con API, SQL Server y ngrok activos.
- Ngrok no reemplaza un hosting permanente ni una base de datos accesible por el servidor desplegado.
- Nunca exponga Swagger, secretos, SQL Server ni endpoints administrativos sin autorizacion.

16. Compilacion del APK

16.1 Proceso reproducible

```
cd JuegoTanquesFront
npm ci
npm run build
npx cap sync android

cd android
.\gradlew.bat assembleDebug
```

El APK debug se genera en JuegoTanquesFront/android/app/build/outputs/apk/debug/app-debug.apk. El artefacto actual distribuible para pruebas tambien se copia como JuegoTanques-mejorado-debug.apk en la raiz del repositorio.

16.2 Firma de produccion

- Crear un keystore privado y respaldarlo fuera del repositorio.
- Configurar signingConfig mediante variables seguras o gradle.properties local.
- Activar minificacion solo despues de validar reglas de Capacitor y SignalR.
- Incrementar versionCode y versionName en cada entrega.
- Generar AAB para Play Store o APK release firmado para distribucion directa.

16.3 Compatibilidad

La configuracion Android usa applicationId com.tanques.juego, minSdk 24, targetSdk 36, compileSdk 36 y version 1.0. La orientacion se controla en tiempo de ejecucion mediante Capacitor Screen Orientation.

17. Pruebas y validacion

17.1 Matriz minima

Area	Prueba	Resultado esperado
Autenticacion	Cerrar y reabrir app	Entra a Inicio sin login mientras el JWT sea valido.
Registro	Codigo correcto/incorrecto/expirado	Crea usuario solo con codigo valido y dentro del limite.
Offline	Entrar sin tocar la pantalla	Equipo activo visible y boton habilitado al terminar la carga.
Ajustes	Volumen 0, vibracion off, sensibilidad extrema	El motor respeta cada valor en offline y online.
Salas	Crear, unir, salir y reconectar	Lista y estado consistentes para todos.
Combate	Disparos, equipos, muerte y ronda	Sin dano aliado y resultados iguales en clientes.
Red	Latencia, perdida breve y ngrok reiniciado	Mensajes claros y reconexion controlada.
Android	Instalacion limpia y actualizacion	Assets, orientacion, audio y haptics operativos.

17.2 Comandos

```
# Frontend
npm run build
npm test

# Backend
dotnet build JuegoTanques.API/JuegoTanques.API.csproj -c Release

# Android
.\gradlew.bat assembleDebug
```

El estado inspeccionado compila en Angular, .NET Release y Android Debug. Existen pruebas spec generadas, pero se recomienda ampliar la cobertura con casos de AuthService, JuegoStore, reglas de SalaServicio y simulaciones de colisiones.

18. Operacion y diagnostico

Sintoma	Causa probable	Accion
Login funciona pero otras llamadas fallan	JWT ausente/expirado o URL API incorrecta.	Revisar tienda-sesion, interceptor, /api/health y hora del dispositivo.
La app no conecta desde datos moviles	Tunel cerrado, URL vieja o backend detenido.	Reiniciar API/ngrok, actualizar environment y recompilar APK.
Sala abre pero no inicia	Host, listos, vidas o estado de ronda inconsistentes.	Revisar eventos SalaActualizada/RondaIniciada y logs del servidor.
Canvas negro	Sala no inicializada, mapa ausente o identidad local sin asociar.	Validar detalle de sala, usuarioId del JWT y mensajes de OnlineGame.
Equipamiento no aparece	Carga asincrona o IDs de catalogo incompatibles.	Verificar Resumen/Presets/Catalogo y mapeo CatalogoUtil.
Sin vibracion	Dispositivo sin soporte, ajuste apagado o plugin no sincronizado.	Activar ajuste, ejecutar cap sync y probar en dispositivo fisico.
Error de copia de DLL	API todavia ejecutandose desde Visual Studio.	Detener depuracion y reconstruir.
Advertencias de paquete vulnerable	Dependencia transitiva desactualizada.	Restaurar NuGet y mantener Pkcs en una version corregida compatible.

18.1 Logs

- Backend: consola de Visual Studio, ILogger y configuracion NLog.
- Frontend web: DevTools, mensajes [API] y [API ERROR].
- Android: Logcat de Android Studio y chrome://inspect para WebView.
- ngrok: inspector local del agente para solicitudes y tiempos de respuesta.

19. Seguridad y recomendaciones

Prioridad	Hallazgo	Recomendacion
Critica	Credenciales SMTP, JWT y conexion presentes en archivos/codigo.	Rotar inmediatamente y migrar a secretos/variables de entorno.
Alta	Controladores CRUD potencialmente publicos.	Aplicar Authorize, politicas por rol y DTO de entrada limitados.
Alta	Estado de salas solo en memoria.	Usar almacenamiento distribuido si se escala a varias instancias.
Alta	JWT de 30 dias sin refresh ni revocacion.	Implementar access/refresh tokens rotatorios y cierre de sesion.
Media	URL ngrok acoplada al interceptor y environments.	Centralizar host y encabezados; usar configuracion por ambiente.
Media	Logs detallados de API en produccion.	Desactivar payloads sensibles y usar niveles por ambiente.
Media	Validacion online parcialmente dependiente del cliente.	Fortalecer autoridad del servidor, rate limiting y anti-replay.
Baja	Archivos/proyectos duplicados o sin uso.	Eliminar solo tras confirmar referencias y compilar toda la solucion.

19.1 Orden de endurecimiento recomendado

- 1. Rotar y retirar todos los secretos del repositorio y del historial Git.
- 2. Proteger controladores y revisar autorizacion por propiedad del recurso.
- 3. Migrar a hosting estable con HTTPS, dominio y base de datos administrada.
- 4. Implementar refresh tokens, revocacion y almacenamiento movil seguro.
- 5. Agregar pruebas automatizadas y observabilidad centralizada.
- 6. Reforzar reglas autoritativas y limites de frecuencia del multijugador.

20. Mantenimiento y evolucion

20.1 Procedimiento de cambio

- Crear rama de trabajo y registrar el objetivo funcional.
- Actualizar contratos backend/frontend de forma compatible.
- Ejecutar build .NET, Angular y Android antes de integrar.
- Probar offline y una partida real con al menos dos dispositivos.
- Actualizar versiones, CHANGELOG y este manual si cambia arquitectura u operacion.
- Generar APK firmado o debug claramente identificado segun el destino.

20.2 Roadmap tecnico sugerido

Horizonte	Objetivos
Corto plazo	Secretos seguros, autorizacion CRUD, limpieza de duplicados, pruebas de autenticacion y salas.
Mediano plazo	Hosting permanente, refresh tokens, migraciones EF, telemetria y pipeline CI/CD.
Largo plazo	Servidor autoritativo, persistencia distribuida de salas, matchmaking, reconexion y anti-cheat.

21. Apendices

A. Puertos y direcciones

Servicio	Direccion de referencia
API local HTTP	http://localhost:5140
API local HTTPS	https://localhost:7123
Health check	/api/health
Hub SignalR	/salasHub
Frontend Ionic	http://localhost:8100
API publica actual	https://hydrant-atonable-broiling.ngrok-free.dev/api (temporal)

B. Catalogo local del motor

Tipo	Codigo	Nombre	Caracteristica
Tanque	TANK-01	Scout Tactico	80 HP, velocidad 9, dash.
Tanque	TANK-02	Juggernaut Pesado	160 HP, blindaje 85, veneno.
Tanque	TANK-03	Vanguardia	110 HP, perforante.
Arma	WEAP-01	Escopeta	Dispersion, alcance corto.
Arma	WEAP-02	Canon de Rebote	Cadencia media.
Arma	WEAP-03	Rifle Pesado	Alta velocidad y baja cadencia.

C. Glosario

Termino	Definicion
JWT	Token firmado usado para identidad y autorizacion.
SignalR	Abstraccion de comunicacion bidireccional en tiempo real.
Hub	Punto de entrada SignalR con metodos invocables por clientes.
EF Core	ORM que mapea entidades .NET a tablas SQL.
Capacitor	Runtime que empaqueta la aplicacion web y expone APIs nativas.
Preset	Combinacion guardada de tanque y arma.
Servidor autoritativo	Servidor que decide el estado valido del juego y no confia en el cliente.

Cierre

Este documento describe el sistema tal como se encuentra en el repositorio al 28 de octubre de 2026. Toda modificacion de endpoints, eventos SignalR, esquema SQL, URL publica o version Android debe reflejarse en una nueva revision del manual.